



# Amazon Managed Streaming for Apache Kafka (Amazon MSK)

## Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

# Amazon MSK



**Amazon Managed Streaming for Apache Kafka (Amazon MSK)** is a fully managed service that enables you to build and run applications that use Apache Kafka to process streaming data.

Amazon MSK utilizes Zookeeper servers.

- *Amazon MSK does not support KRaft*

There are two types of nodes:

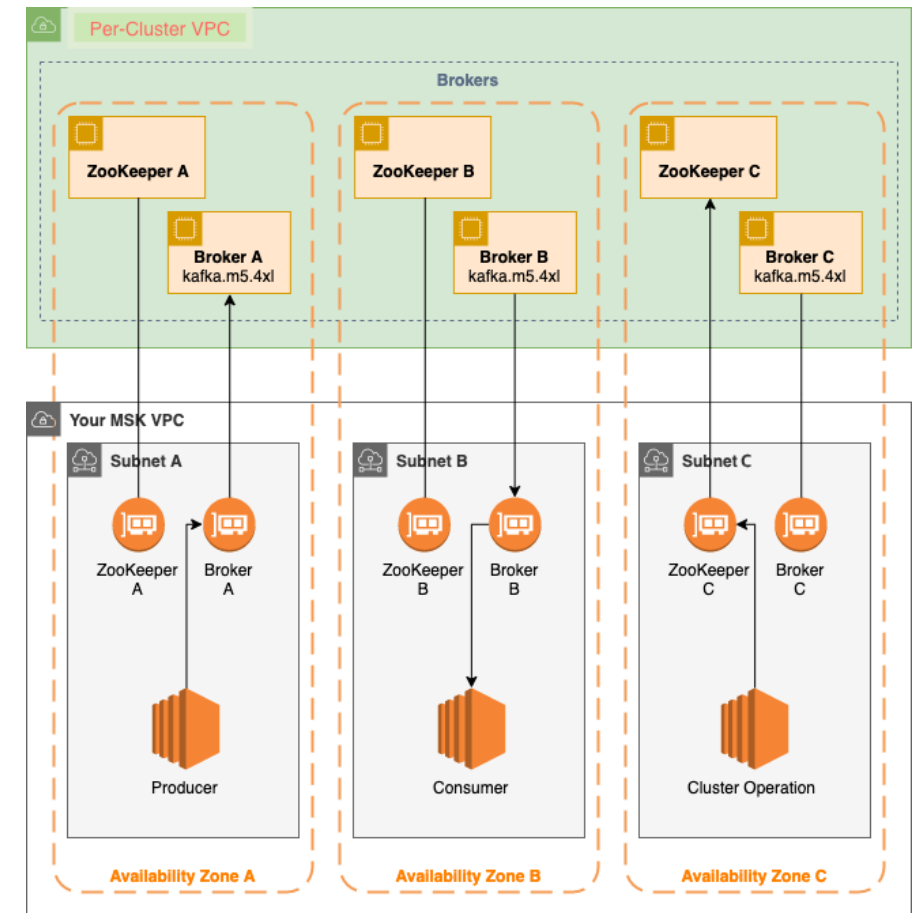
- Broker nodes — handles storage and processing of messages
- ZooKeeper nodes (Znodes) — manages the overall structure of the cluster

Amazon MSK comes in two types of clusters:

- **Provisioned** — you manage the broker instances
- **Serverless** — you pay for what you use, and you don't have to manage instances

Kafka has direct integrations with

- Amazon S3
- EventBridge Pipes



# Amazon MSK

Amazon MSK is launched within your VPC and need your connect to originate from the same VPC. You can enable public access on clusters afte the launch of a cluster

Turn on **public access** for an MSK cluster



```
aws kafka update-connectivity \  
--cluster-arn ClusterArn \  
--current-version Current-Cluster-Version \  
--connectivity-info '{"PublicAccess": {"Type": "SERVICE_PROVIDED_EIPS"}}'
```

# Amazon MSK – Bootstrap Brokers

**Bootstrap brokers** refers to a list of brokers **endpoints** that an Apache Kafka client can use as a **starting point to connect to the cluster**.

```
aws kafka get-bootstrap-brokers --cluster-arn ClusterArn
# {
#   "BootstrapBrokerStringPublicSaslIam": "...",
#   "BootstrapBrokerStringSaslIam": "...",
# }
```

- **BootstrapBrokerStringPublicSaslIam** is for public access
- **BootstrapBrokerStringSaslIam** string for access from within AWS.

Depending on how many AZs you've deployed your cluster in, there will be a comma separated list with an endpoint for each AZ.

# Amazon MSK – Zookeeper Connection String

**ZooKeeper connection string URL** is used with Kafka to specify the host and port of the ZooKeeper ensemble that Kafka should connect to for managing cluster metadata and coordination:

- Broker Registration
- **Topic Configuration**
- Cluster Membership
- Quota Management
- Access Control Lists (ACLs)



```
bin/kafka-topics.sh --create \  
--zookeeper "ZookeeperConnectionString" \  
--replication-factor 2 \  
--partitions 1 \  
--topic my-topic
```

We can get the **Zookeeper Connection String URL** using **describe-cluster**



```
aws kafka describe-cluster \  
--cluster-arn ClusterArn \  
--query ClusterInfo.ZookeeperConnectionString
```

*We cannot call the describe-clusters against MSK Serverless Cluster*

Starting from Kafka 2.8, there is a mode called KRaft (Kafka Raft Metadata mode) which allows Kafka to run without ZooKeeper.

# Amazon MSK Connect

**MSK Connect** is a feature of Amazon MSK that makes it easy for developers to stream data to and from their Apache Kafka clusters.

MSK Connect uses **Kafka Connect open-source framework** for connecting Apache Kafka clusters with external systems such as databases, search indexes, and file systems

You can download Kafka Connect plugins (or create your own) Upload to s3 and then create a plugin in MSK Connect

```
aws kafkaconnect create-custom-plugin \  
--name mongodb \  
--description "MongoDB Kafka Connector" \  
--content-location '{  
  "s3Location":{  
    "bucketArn": "<arn-of-your-s3-bucket>",  
    "fileKey": "mongo-kafka-connector.jar"  
  }  
'
```

Then you create a connector specifying the plugin and configuration information to your source

```
aws kafkaconnect create-connector \  
--kafka-cluster clusterArn=<kafka-cluster-arn> \  
--kafka-cluster-client-authentication-type NONE \  
--kafka-cluster-encryption-in-transit-type PLAINTEXT \  
--plugin name=<plugin-name>,revision=<plugin-revision> \  
--service-execution-role-arn <your-execution-role-arn> \  
--connector-configuration '{  
  {  
    "name": "MongoDBSourceConnector",  
    "config": {  
      "connector.class": "com.mongodb.kafka.connect.MongoSourceConnector",  
      "tasks.max": "1",  
      "topics": "mongoDBTopic",  
      "connection.uri": "mongodb://yourMongoDBInstance",  
      // other necessary configuration properties  
    }  
  }  
'
```

# What is Apache Kafka?



Apache Kafka is an **open-source streaming platform** to create **high-performance data pipelines**, streaming analytics, data integration, and mission-critical applications.

Kafka was originally developed by LinkedIn, and open-sourced in 2011



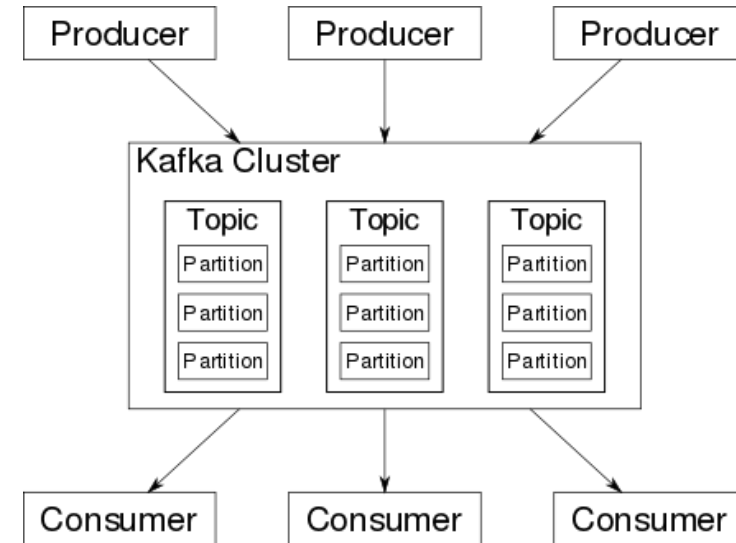
Kafka was written in Scala and Java.  
to use Kafka, you need to write **Java** code.

In Kafka data is stored in partitions on a Kafka Cluster which can span multiple machines (distributed computing)

**Producers** publish messages in a key and value format using the Kafka Producer API

**Consumers** can listen for messages and consume them using the Kafka Consumer API

Messages are organized into **Topics**. Producers will push messages to topics and consumers will listen on topics.



- We can interact with Kafka using Kafka-CLI scripts
- We can use a programming SDK for Kafka in various languages

# What is Apache Zookeeper?

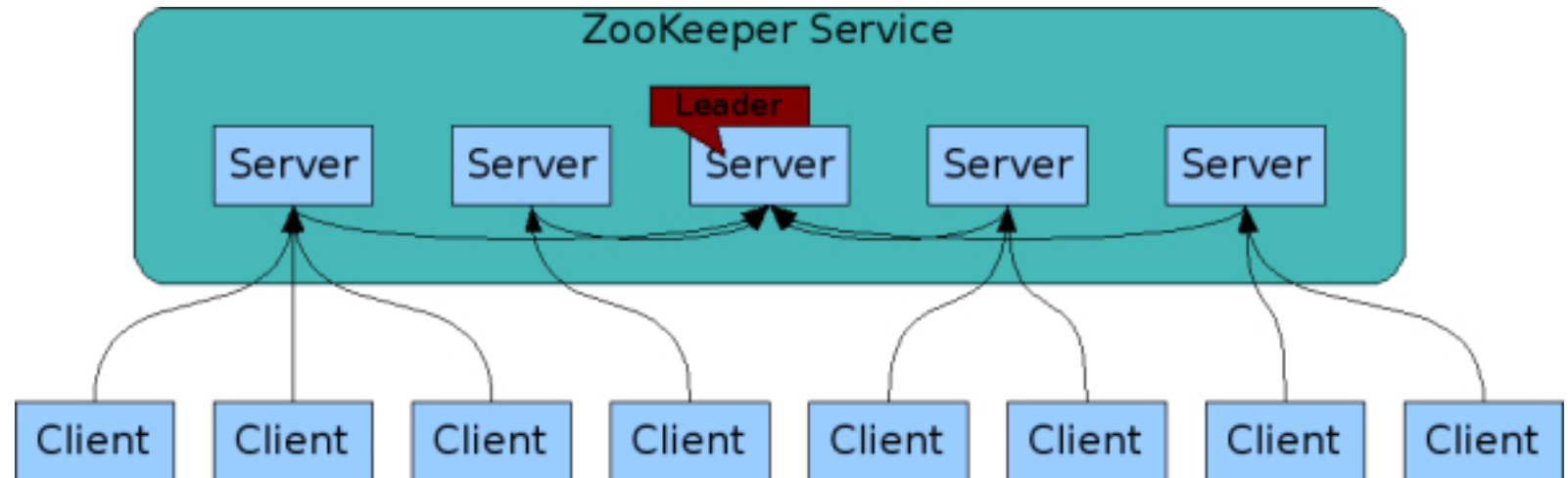


**Apache ZooKeeper** is an **open-source server** for highly reliable distributed coordination of cloud applications

Open-source projects that use Zookeeper:

- Apache Hadoop
- Apache Kafka
- Apache Solr
- Apache Hbase
- Apache Accumulo
- Apache Druid
- Apache Helix

Zookeeper exposes common services - such as naming, configuration management, synchronization, and group services - in a simple interface so you don't have to write them from scratch



# Kafka CLI

The Kafka CLI scripts are **downloaded alongside Kafka**.

<https://kafka.apache.org/downloads>

Kafka CLI is different from other CLI tools in that it's a series of scripts instead of one binary.

```
bin /
— connect-distributed.sh
— connect-mirror-maker.sh
— connect-plugin-path.sh
— connect-standalone.sh
— kafka-acls.sh
— kafka-broker-api-versions.sh
— kafka-client-metrics.sh
— kafka-cluster.sh
— kafka-configs.sh
— kafka-console-consumer.sh
— kafka-console-producer.sh
— kafka-consumer-groups.sh
— kafka-consumer-perf-test.sh
— kafka-delegation-tokens.sh
— kafka-delete-records.sh
— kafka-dump-log.sh
— kafka-e2e-latency.sh
— kafka-features.sh
— kafka-get-offsets.sh
— kafka-jmx.sh
— kafka-leader-election.sh
— kafka-log-dirs.sh
— kafka-metadata-quorum.sh
— kafka-metadata-shell.sh
— kafka-mirror-maker.sh
— kafka-producer-perf-test.sh
— kafka-reassign-partitions.sh
— kafka-replica-verification.sh
— kafka-run-class.sh
— kafka-server-start.sh
— kafka-server-stop.sh
— kafka-storage.sh
— kafka-streams-application-reset.sh
— kafka-topics.sh
— kafka-transactions.sh
— kafka-verifiable-consumer.sh
— kafka-verifiable-producer.sh
— trogdor.sh
— zookeeper-security-migration.sh
— zookeeper-server-start.sh
— zookeeper-server-stop.sh
— zookeeper-shell.sh
```



Kafka's testing framework is called Trogdor a reference to Homestar Runner character

# Kafka CLI

## Creating a **Topic**

```
./bin/kafka-topics.sh \  
--create --topic my-topic \  
--bootstrap-server <broker1-hostname:port> \  
--replication-factor 3 \  
--partitions 1
```

## Sending Messages to a **Producer**

```
# Waits for input, each line will be sent as a message  
./bin/kafka-console-producer.sh \  
--broker-list <broker1-hostname:port>,<broker2-hostname:port> \  
--topic my-topic  
  
# Send a single message  
echo "Hello, Kafka!" | ./bin/kafka-console-producer.sh \  
--broker-list <broker1-hostname:port> \  
--topic my-topic  
  
# Send multiple messages by providing a multiple line file with  
each message on its own line  
cat messages.txt | ./bin/kafka-console-producer.sh \  
--broker-list <broker1-hostname:port> \  
--topic my-topic
```

## Receive Messages from a Topic as a **Consumer**

```
./bin/kafka-console-consumer.sh \  
--bootstrap-server <broker1-hostname:port> \  
--topic my-topic \  
--from-beginning
```